

Ejercicio 1 – Clase y objeto: Empleado

Análisis esperado

Clase:

None
Empleado

Atributos:

None
Nombre
DNI
Salario

Método:

None
MostrarDatos()

Implementación posible

```
None
public class Empleado
{
    public string Nombre { get; set; }
    public string DNI { get; set; }
    public decimal Salario { get; set; }

    public void MostrarDatos()
    {
        Console.WriteLine($"Nombre: {Nombre}");
        Console.WriteLine($"DNI: {DNI}");
        Console.WriteLine($"Salario: ${Salario}");
    }
}
```

Uso

None

```
Empleado empleado = new Empleado();
```

```
empleado.Nombre = "Juan Perez";
```

```
empleado.DNI = "30123456";
```

```
empleado.Salario = 850000;
```

```
empleado.MostrarDatos();
```

Conceptos que deberían aparecer

- Clase.
- Objeto.
- Propiedades/atributos.
- Método.
- Instanciación con `new`.

No hace falta definir constructor por el momento.

Ejercicio 2 – Encapsulamiento: Cuenta bancaria

Este ejercicio es especialmente importante porque permite evaluar si realmente entendieron **encapsulamiento**.

Análisis

Clase:

None

Cuenta

Atributos:

None

Numero

Titular

Saldo

Pero **Saldo** no debería poder modificarse directamente desde afuera.

Métodos:

```
None
Depositar()
Retirar()
ConsultarSaldo()
```

Implementación modelo

```
None
public class Cuenta
{
    public string Numero { get; set; }
    public string Titular { get; set; }

    private decimal saldo;

    public void Depositar(decimal monto)
    {
        if (monto > 0)
        {
            saldo += monto;
        }
    }

    public bool Retirar(decimal monto)
    {
        if (monto <= 0)
        {
            return false;
        }

        if (monto > saldo)
        {
            return false;
        }

        saldo -= monto;
        return true;
    }
}
```

```
    }

    public decimal ConsultarSaldo()
    {
        return saldo;
    }
}
```

Uso

None

```
Cuenta cuenta = new Cuenta();

cuenta.Numero = "001";
cuenta.Titular = "Juan Perez";

cuenta.Depositar(100000);

bool resultado = cuenta.Retirar(30000);

Console.WriteLine($"Saldo: {cuenta.ConsultarSaldo()}");
```

¿Qué debería detectar el alumno?

Esto:

None

```
cuenta.saldo = -50000;
```

no debería ser posible porque `saldo` es `private`.

Y tampoco debería existir algo como:

None

```
cuenta.Saldo = -50000;
```

si el diseño permite modificar el saldo libremente.

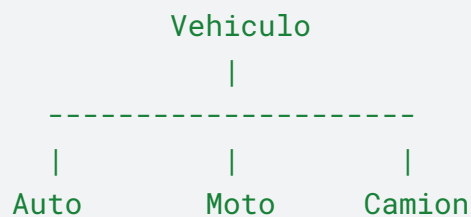
Concepto central

Encapsulamiento: el objeto controla cómo se modifica su estado interno.

Ejercicio 3 – Abstracción: Vehículos

Modelo conceptual

None



Clase abstracta

None

```
public abstract class Vehiculo
{
    public string Patente { get; set; }
    public string Marca { get; set; }
    public string Modelo { get; set; }

    public abstract void Arrancar();
}
```

Auto

None

```
public class Auto : Vehiculo
{
    public override void Arrancar()
    {
        Console.WriteLine("El auto arranca utilizando su motor.");
    }
}
```

Moto

None

```
public class Moto : Vehiculo
{
    public override void Arrancar()
    {
        Console.WriteLine("La moto arranca.");
    }
}
```

Camión

None

```
public class Camion : Vehiculo
{
    public override void Arrancar()
    {
        Console.WriteLine("El camión arranca.");
    }
}
```

Lo importante

No es necesario que la implementación del método sea exactamente esa.

Lo que deberías comprender es que:

None

```
Vehiculo
```

representa una **abstracción general**.

No necesitamos crear:

None

```
new Vehiculo();
```

porque `Vehiculo` representa un concepto general.

En cambio:

None

```
new Auto();  
new Moto();  
new Camion();
```

sí representan objetos concretos.

Ejercicio 4 – Herencia

Partimos del ejercicio anterior.

Jerarquía esperada

None

```
Vehiculo  
|  
+---- Auto  
|  
+---- Moto  
|  
+---- Camion
```

Implementación

None

```
public abstract class Vehiculo  
{  
    public string Patente { get; set; }  
    public string Marca { get; set; }  
    public string Modelo { get; set; }  
  
    public abstract void Arrancar();  
}
```

None

```
public class Auto : Vehiculo  
{
```

```
        public int CantidadPuertas { get; set; }

        public override void Arrancar()
        {
            Console.WriteLine("Arrancando auto...");
        }
    }
}
```

None

```
public class Moto : Vehiculo
{
    public int Cilindrada { get; set; }

    public override void Arrancar()
    {
        Console.WriteLine("Arrancando moto...");
    }
}
```

None

```
public class Camion : Vehiculo
{
    public double CapacidadCarga { get; set; }

    public override void Arrancar()
    {
        Console.WriteLine("Arrancando camión...");
    }
}
```

¿Qué debería poder explicar el alumno?

Que:

None

```
Auto ES UN Vehiculo
Moto ES UN Vehiculo
```

Camion ES UN Vehiculo

Por eso:

None
`public class Auto : Vehiculo`

es correcto.

En cambio:

None
`public class Motor : Auto`

no tiene sentido para este modelo.

Ejercicio 5 – Polimorfismo

Este es uno de los ejercicios más importantes.

Código esperado

None

```
List<Vehiculo> vehiculos = new List<Vehiculo>();

vehiculos.Add(new Auto());
vehiculos.Add(new Moto());
vehiculos.Add(new Camion());

foreach (Vehiculo vehiculo in vehiculos)
{
    vehiculo.Acelerar();
}
```

Las clases podrían ser:

None

```
public abstract class Vehiculo
{
    public string Patente { get; set; }

    public abstract void Acelerar();
}
```

None

```
public class Auto : Vehiculo
{
    public override void Acelerar()
    {
        Console.WriteLine("El auto acelera progresivamente.");
    }
}
```

None

```
public class Moto : Vehiculo
{
    public override void Acelerar()
    {
        Console.WriteLine("La moto acelera rápidamente.");
    }
}
```

None

```
public class Camion : Vehiculo
{
    public override void Acelerar()
    {
        Console.WriteLine("El camión acelera lentamente.");
    }
}
```

Lo fundamental

La variable:

None

```
Vehiculo vehiculo
```

puede referenciar diferentes tipos concretos.

Cuando hacemos:

None

```
vehiculo.Acelerar();
```

C# ejecuta la implementación correspondiente al objeto real.

Por ejemplo:

None

```
Auto      → El auto acelera progresivamente.  
Moto      → La moto acelera rápidamente.  
Camion    → El camión acelera lentamente.
```

Error conceptual a prestar atencion

Algo como:

None

```
if (vehiculo is Auto)  
{  
    ...  
}  
else if (vehiculo is Moto)  
{  
    ...  
}
```

para decidir cómo acelerar.

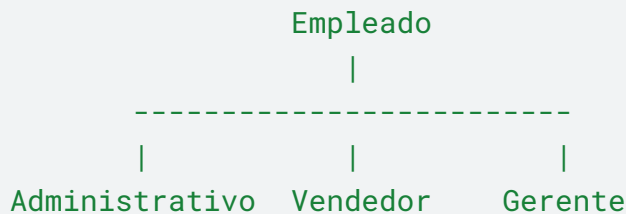
No necesariamente es un error sintáctico, pero **contradice el objetivo pedagógico del ejercicio**, porque justamente queremos aprovechar el polimorfismo.

Ejercicio 6 – Empleados + polimorfismo

Este es un excelente ejercicio integrador.

Modelo

None



Clase base

None

```
public abstract class Empleado
{
    public string Nombre { get; set; }
    public string DNI { get; set; }
    public decimal SalarioBasico { get; set; }

    public abstract decimal CalcularSalario();
}
```

Administrativo

None

```
public class Administrativo : Empleado
{
    public override decimal CalcularSalario()
    {
        return SalarioBasico;
    }
}
```

Vendedor

None

```
public class Vendedor : Empleado
```

```

{
    public decimal Comision { get; set; }

    public override decimal CalcularSalario()
    {
        return SalarioBasico + Comision;
    }
}

```

Gerente

None

```

public class Gerente : Empleado
{
    public decimal Bono { get; set; }

    public override decimal CalcularSalario()
    {
        return SalarioBasico + Bono;
    }
}

```

Programa principal

None

```

List<Empleado> empleados = new List<Empleado>();

empleados.Add(new Administrativo
{
    Nombre = "Ana",
    DNI = "11111111",
    SalarioBasico = 500000
});

empleados.Add(new Vendedor
{
    Nombre = "Pedro",
    DNI = "22222222",
    SalarioBasico = 500000,

```

```

        Comision = 100000
    });

empleados.Add(new Gerente
{
    Nombre = "Laura",
    DNI = "33333333",
    SalarioBasico = 800000,
    Bono = 200000
});

foreach (Empleado empleado in empleados)
{
    Console.WriteLine(
        $"{empleado.Nombre}: ${empleado.CalcularSalario()}"
    );
}

```

Resultado esperado

```

None
Ana: $500000
Pedro: $600000
Laura: $1000000

```

Conceptos evaluables

Acá deberían aparecer claramente:

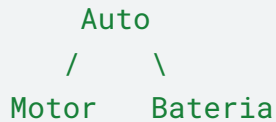
- Abstracción.
- Herencia.
- Polimorfismo.
- `abstract`.
- `override`.
- Colecciones.
- Objetos de diferentes clases almacenados como `Empleado`.

Ejercicio 7 – Composición: Auto y Motor

Este ejercicio tiene como objetivo que los alumnos **no utilicen herencia incorrectamente**.

Modelo correcto

None



También podríamos tener:

None



Motor

None

```
public class Motor
{
    public string Modelo { get; set; }

    public void Arrancar()
    {
        Console.WriteLine("Motor arrancando...");
    }

    public void Detener()
    {
        Console.WriteLine("Motor detenido.");
    }
}
```

Batería

None

```
public class Bateria
{
    public int Capacidad { get; set; }
}
```

Auto

None

```
public class Auto
{
    public string Marca { get; set; }

    public Motor Motor { get; set; }

    public Bateria Bateria { get; set; }

    public void Arrancar()
    {
        Motor.Arrancar();
    }

    public void Detener()
    {
        Motor.Detener();
    }
}
```

Uso

None

```
Motor motor = new Motor();

Bateria bateria = new Bateria
{
    Capacidad = 70
};

Auto auto = new Auto
{
    Marca = "Toyota",
    Motor = motor,
    Bateria = bateria
};

auto.Arrancar();
```

Concepto fundamental

El alumno debería poder explicar:

None

Auto TIENE un Motor.
Auto TIENE una Bateria.

No:

None

Auto ES un Motor.

Por lo tanto:

None

```
public class Auto : Motor
```

sería un mal diseño para este requerimiento.

Ejercicio 8 – Integrador: Biblioteca

Este es el ejercicio donde hay varias formas válidas de modelarlo.

Análisis esperado

Sustantivos

Del requerimiento podemos obtener:

None

Biblioteca
Sistema
Libro
ISBN
Título
Autor
Año
Socio

```
DNI
Nombre
Número de socio
Préstamo
```

No todos se convierten en clases.

Clases principales

Una solución razonable:

```
None
Libro
Socio
Prestamo
Biblioteca
```

Aunque también sería aceptable que algunos alumnos modelen `Autor` como clase, **si pueden justificarlo**.

Clase Libro

```
None
public class Libro
{
    public string ISBN { get; set; }
    public string Titulo { get; set; }
    public string Autor { get; set; }
    public int AnioPublicacion { get; set; }

    private bool prestado;

    public bool EstaPrestado()
    {
        return prestado;
    }

    public bool Prestar()
```

```

    {
        if (prestado)
        {
            return false;
        }

        prestado = true;
        return true;
    }

    public void Devolver()
    {
        prestado = false;
    }
}

```

Clase Socio

```

None
public class Socio
{
    public string DNI { get; set; }
    public string Nombre { get; set; }
    public int NumeroSocio { get; set; }
}

```

Clase Prestamo

Una posible solución:

```

None
public class Prestamo
{
    public Socio Socio { get; set; }

    public Libro Libro { get; set; }
}

```

```

    public DateTime Fecha { get; set; }

    public Prestamo(Socio socio, Libro libro)
    {
        Socio = socio;
        Libro = libro;
        Fecha = DateTime.Now;
    }
}

```

Esto permite representar:

```

None
Prestamo
|
+----- Socio
|
+----- Libro

```

Y acá aparece una idea muy importante:

Prestamo no necesita heredar de Socio ni de Libro.

Es una clase que **relaciona** ambas entidades.

Clase Biblioteca

Una posible implementación:

```

None
public class Biblioteca
{
    private List<Libro> libros = new List<Libro>();

    private List<Socio> socios = new List<Socio>();

    private List<Prestamo> prestamos = new List<Prestamo>();
}

```

```

    public void RegistrarLibro(Libro libro)
    {
        libros.Add(libro);
    }

    public void RegistrarSocio(Socio socio)
    {
        socios.Add(socio);
    }

    public bool PrestarLibro(Socio socio, Libro libro)
    {
        if (!libro.Prestar())
        {
            return false;
        }

        Prestamo prestamo = new Prestamo(socio, libro);

        prestamos.Add(prestamo);

        return true;
    }

    public void DevolverLibro(Libro libro)
    {
        libro.Devolver();
    }
}

```

Prueba del ejercicio 8

None

```
Biblioteca biblioteca = new Biblioteca();
```

```
Socio socio1 = new Socio
```

```
{
    DNI = "30123456",
    Nombre = "Juan Perez",
    NumeroSocio = 1
};

Libro libro1 = new Libro
{
    ISBN = "978-1234567890",
    Titulo = "Programación en C#",
    Autor = "Autor Ejemplo",
    AnioPublicacion = 2024
};

biblioteca.RegistrarSocio(socio1);
biblioteca.RegistrarLibro(libro1);

bool resultado1 =
    biblioteca.PrestarLibro(socio1, libro1);

Console.WriteLine(resultado1);
```

Resultado:

None
True

Intentamos nuevamente:

```
None
bool resultado2 =
    biblioteca.PrestarLibro(socio1, libro1);

Console.WriteLine(resultado2);
```

Resultado:

None

False

Porque el libro ya estaba prestado.

Después:

None

```
biblioteca.DevolverLibro(libro1);
```

Y nuevamente:

None

```
bool resultado3 =  
    biblioteca.PrestarLibro(socio1, libro1);  
  
Console.WriteLine(resultado3);
```

Resultado:

None

True